API. GA7-220501096-AA5-EV02.

Marco Aníbal Esguerra Díaz

(3186647) Análisis y Desarrollo de Software

Arnaldo Alfonso Montiel Brun

Centro de Comercio, Industria y

Turismo Regional Córdoba

Servicio Nacional de Aprendizaje (SENA)

29 de Junio del 2026

Introducción

El desarrollo tecnológico de la plataforma de comercio electrónico Tropos ha transitado de manera progresiva por un riguroso ciclo de ingeniería de software, partiendo de una fase puramente estética y de maquetación estática en el cliente hacia la consolidación de un sistema transaccional robusto y dinámico con persistencia relacional de datos. En las fases iniciales, la interfaz del proyecto dependía exclusivamente de la rigidez estructural de HTML5 y las propiedades de diseño de hojas de estilo CSS3 aplicadas bajo el modelo de maquetación de caja flexible. Si bien estas interfaces garantizaban una experiencia visual minimalista de alta fidelidad, la lógica de negocio se encontraba limitada técnicamente por la ausencia de un backend capaz de procesar datos, interactuar con bases de datos relacionales y gestionar sesiones seguras del lado del servidor.

Para superar el paradigma de un sitio web de catálogo pasivo y transformarlo en una verdadera aplicación web escalable, segura y adaptable, se procedió al diseño y codificación de una capa de servicios web. Esta capa de servicios, estructurada en el lenguaje de servidor PHP 8 en un entorno local configurado mediante la pila de servicios XAMPP (Apache y MySQL), actúa como el núcleo transaccional de Tropos. El objetivo primordial de este manual es documentar de forma exhaustiva la arquitectura lógica, la especificación de los endpoints construidos, el modelo de datos subyacente y los mecanismos de seguridad integrados, tales como las sentencias preparadas de MySQLi y el procesamiento híbrido del cuerpo de peticiones. Asimismo, se detalla la resolución de los problemas nominales en la base de datos y las advertencias de control de peticiones identificados durante la etapa de pruebas, sirviendo este documento como el mapa técnico de referencia del backend de Tropos para futuros despliegues o integraciones con frameworks modernos de frontend.

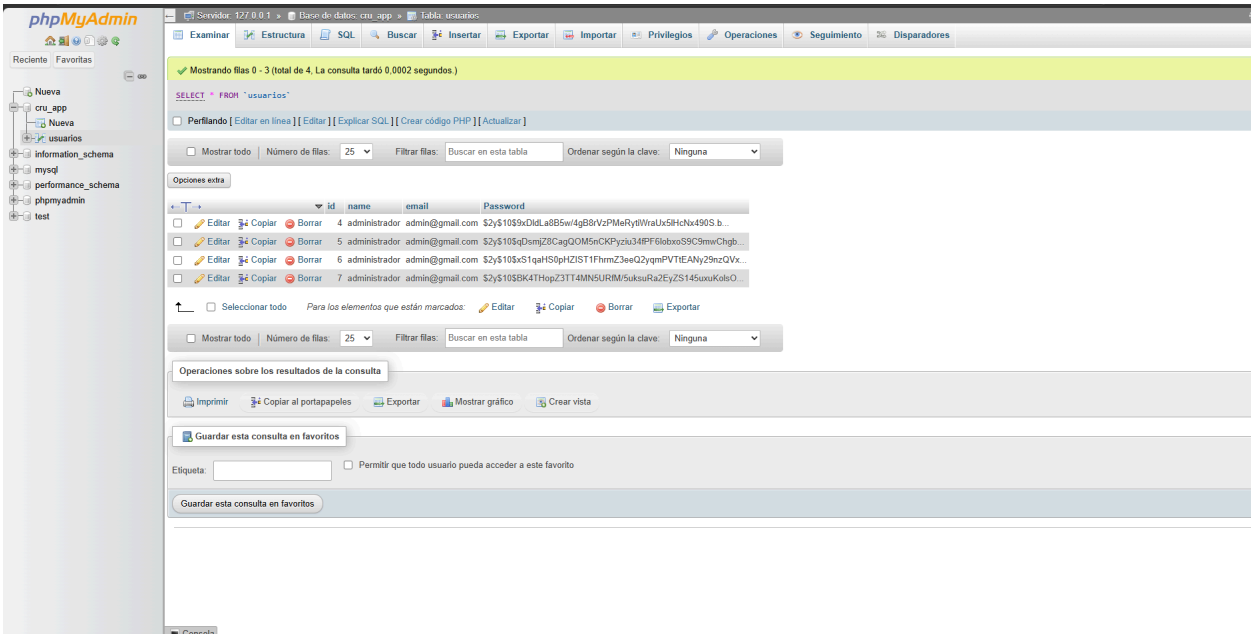
Arquitectura General del Sistema y Persistencia de Datos

La arquitectura lógica adoptada para la plataforma Tropos sigue un modelo cliente-servidor clásico donde las peticiones del cliente son procesadas de manera asíncrona o mediante formularios codificados hacia un servidor de aplicaciones local Apache. El flujo transaccional se sustenta sobre tres componentes de software fundamentales e independientes que garantizan la modularidad, la mantenibilidad del código y el desacoplamiento de responsabilidades: la base de datos, el conector de datos y los scripts de procesamiento lógico.

La persistencia del sistema está delegada al motor relacional MySQL. Para las actividades transaccionales de control de acceso y registro de administradores, se utiliza la base de datos local denominada crud_app. Dentro de este esquema relacional, la tabla de mayor relevancia técnica es la tabla de usuarios, cuya estructura física ha sido optimizada para almacenar de manera segura los registros de los administradores y futuros clientes del ecosistema comercial.

La tabla de usuarios está compuesta por los siguientes campos críticos en el motor:

- **id:** Tipo de dato entero, autoincremental, configurado como llave primaria para garantizar la unicidad de cada registro almacenado.
- **name:** Tipo varchar, configurado para almacenar el nombre del usuario o administrador del sistema.
- **email:** Tipo varchar, definido con una restricción única para evitar que existan dos cuentas registradas con la misma dirección de correo electrónico en la plataforma.
- **password:** Tipo varchar de alta longitud, destinado a almacenar la firma criptográfica resultante del cifrado de la clave del usuario.



Estándares de Codificación y Medidas de Seguridad

La codificación de la API de Tropos se rige por altos estándares de calidad de software y directrices de seguridad informática orientadas a mitigar las vulnerabilidades más comunes del Top 10 de OWASP. Durante la escritura del código PHP, se priorizó la prevención de inyección SQL a través del uso mandatorio del controlador nativo MySQLi con sentencias preparadas (*Prepared Statements*). Al parametrizar las consultas con comodines de interrogación y asociar los tipos de datos de entrada mediante variables tipadas, se evita la alteración maliciosa de la estructura de las consultas enviadas al motor MySQL.

En el aspecto de la seguridad criptográfica, se ha descartado de plano el almacenamiento de contraseñas en texto plano o el uso de algoritmos vulnerables y obsoletos como MD5 o SHA1. En su lugar, el backend utiliza la función nativa password_hash() con el algoritmo robusto BCRYPT, el cual genera automáticamente un valor de sal (*salt*) aleatorio por cada contraseña procesada, entregando una cadena de caracteres irreversible que es almacenada de forma

segura en la base de datos. Para la posterior validación del inicio de sesión, el sistema aplica la función password_verify(), comparando la clave entrante con el hash almacenado sin necesidad de descifrar la clave original, blindando así los datos ante accesos no autorizados a la base de datos.

Por último, el control de accesos se gestiona del lado del servidor mediante el protocolo nativo de sesiones de PHP (session_start()). Esto permite generar identificadores únicos de sesión que se almacenan de manera segura en el servidor y se asocian al navegador del usuario a través de cookies de sesión temporales, restringiendo el ingreso a las páginas administrativas (como inicio.php) únicamente a aquellos clientes que hayan superado de manera satisfactoria el proceso de login.

Especificación Detallada de los Endpoints

A continuación, se presenta la especificación técnica de los servicios web que componen la API del proyecto Tropos, detallando su estructura sintáctica, métodos de envío, parámetros obligatorios y las respuestas estandarizadas del servidor.

Conexión y Persistencia Base de Datos

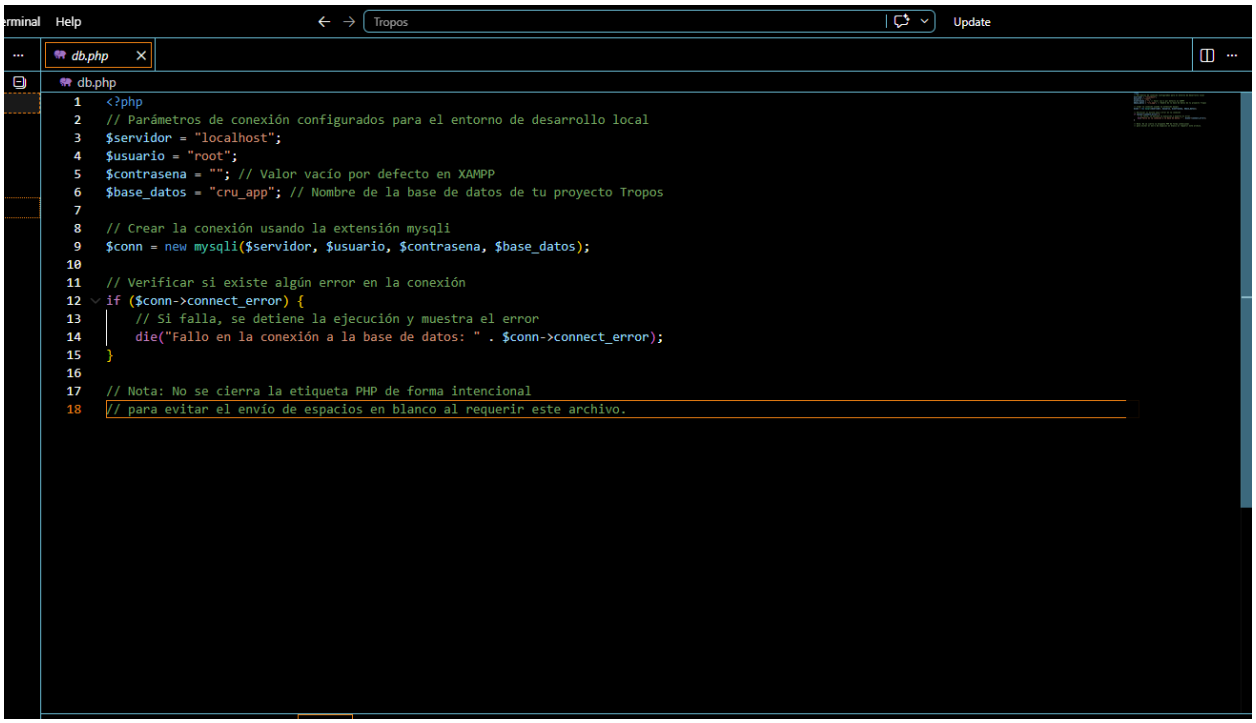
- **Archivo Local:** db.php
- **Método Operativo:** Inicialización interna de conexión MySQLi.
- **Descripción:** Este componente establece la conexión inmutable, persistente y parametrizada con el motor de base de datos local MySQL. Incorpora un control de excepciones estricto que aborta de inmediato la ejecución del hilo y devuelve el error del

socket de base de datos en caso de una falla física de comunicación, impidiendo el procesamiento de scripts huérfanos.

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "crud_app";

// Crear conexión mysqli utilizando el controlador integrado
$conn = new mysqli($servername, $username, $password, $dbname);

// Validar si el canal de conexión falló
if ($conn->connect_error) {
    die("Conexión fallida físicamente: " . $conn->connect_error);
}
?>
```



Servicio de Inicialización de Administrador

- **URL del Endpoint:** http://localhost/Project_Tropos/Tropos/crear_admin.php
- **Método de Petición:** POST o acceso directo de inicialización del sistema.
- **Descripción:** Este script registra un usuario administrador maestro en la base de datos, convirtiendo la clave plana en una firma hash irreversible. Fue fundamental para poblar la base de datos de pruebas que previamente se encontraba vacía.
- **Parámetros del Body:** Ninguno requerido por ser un script autoejecutable estructurado con variables seguras preestablecidas, aunque adaptable para recibir un objeto JSON.

<?php

require 'db.php';

```
$nombre = 'administrador';
$email = 'admin@gmail.com';
$password = password_hash('123456', PASSWORD_DEFAULT);

// Corrección de correspondencia nominal de columnas en SQL
$sql = "INSERT INTO usuarios (name, email, password) VALUES (?, ?, ?)";
$stmt = $conn->prepare($sql);

if (!$stmt) {
    die("Error crítico en la consulta SQL: " . $conn->error);
}

$stmt->bind_param('sss', $nombre, $email, $password);

if ($stmt->execute()) {
    echo "Administrador creado correctamente";
} else {
    echo "Error al ejecutar el guardado: " . $stmt->error;
}
?>
```

crear_admin.php

```php
<?php
// Requerimos el archivo de conexión a la base de datos (db.php)
require 'db.php';

// Definimos los datos del primer usuario administrador
$nombre = 'administrador';
$email = 'admin@gmail.com';

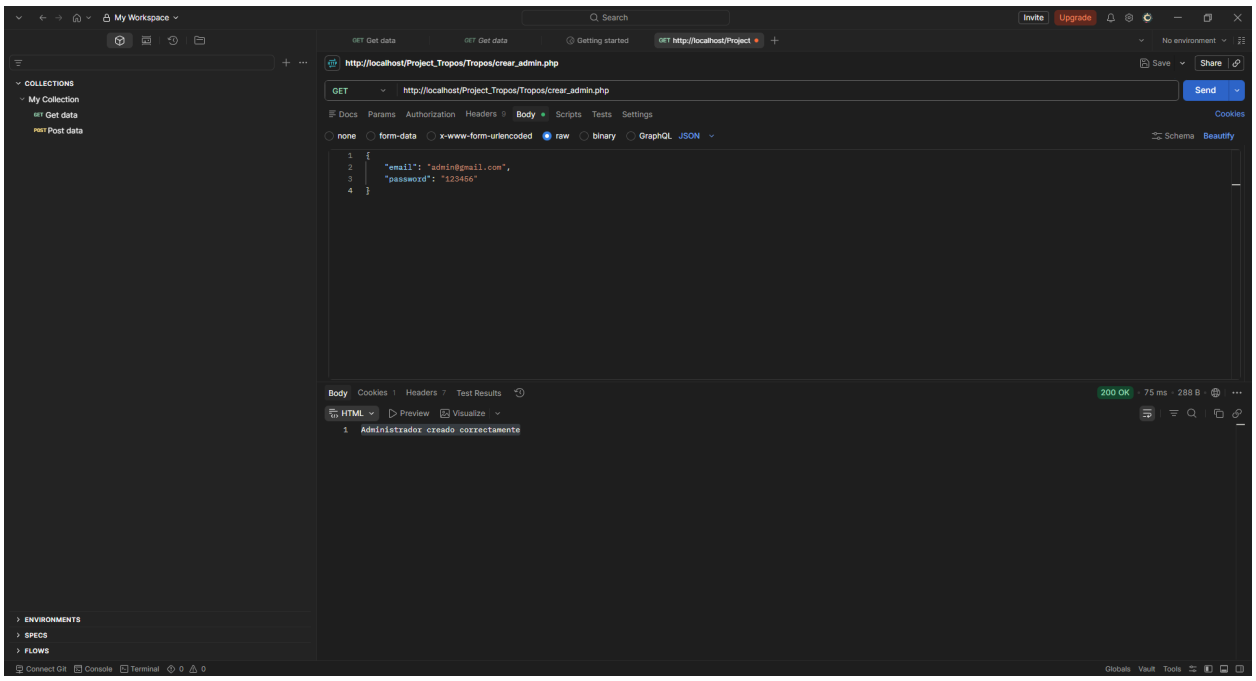
// Encriptamos la contraseña ('123456') usando password_hash por seguridad
$password = password_hash('123456', PASSWORD_DEFAULT);

// Corrección: Se cambia 'nombre' por 'name' para que coincida con tu base de datos física
$sql = "INSERT INTO usuarios (name, email, password) VALUES (?, ?, ?)";
$stmt = $conn->prepare($sql);

// Validación de seguridad para atrapar el fallo de MySQL y evitar que colapse Postman
if (!$stmt) {
    die("Error crítico en la consulta SQL: " . $conn->error);
}

// Asociamos los 3 parámetros tipo string ('sss')
$stmt->bind_param('sss', $nombre, $email, $password);

// Ejecutamos la consulta y verificamos si se guardó correctamente
if ($stmt->execute()) {
    echo "Administrador creado correctamente";
} else {
    echo "Error al ejecutar el guardado: " . $stmt->error;
}
?>
```



Servicio de Autenticación de Usuario (Login)

- **URL del Endpoint:** http://localhost/Project_Tropos/Tropos/login.php
- **Método de Petición:** POST
- **Descripción:** Valida de forma segura el acceso de los administradores. Cuenta con soporte de recepción híbrido para procesar datos enviados de forma clásica (Form-urlencoded) o estructurada de forma raw (JSON), devolviendo el control al cliente Postman sin generar advertencias (*warnings*).
- **Parámetros de Entrada (JSON en el Body de la petición):**

```
{
   "email": "admin@gmail.com",
   "password": "123456"
}
```

```
<?php
session_start();
require 'db.php';

$email = $_POST['email'] ?? null;
$password = $_POST['password'] ?? null;

// Soporte híbrido para peticiones RAW JSON enviadas desde Postman
if (!$email || !$password) {
   $raw_input = file_get_contents('php://input');
```

```
    $json_data = json_decode($raw_input, true);
    if ($json_data) {
        $email = $json_data['email'] ?? $email;
        $password = $json_data['password'] ?? $password;
    }
}

if (!$email || !$password) {
    echo "Error en Postman: No se recibieron las llaves obligatorias ('email' y 'password').";
    exit();
}

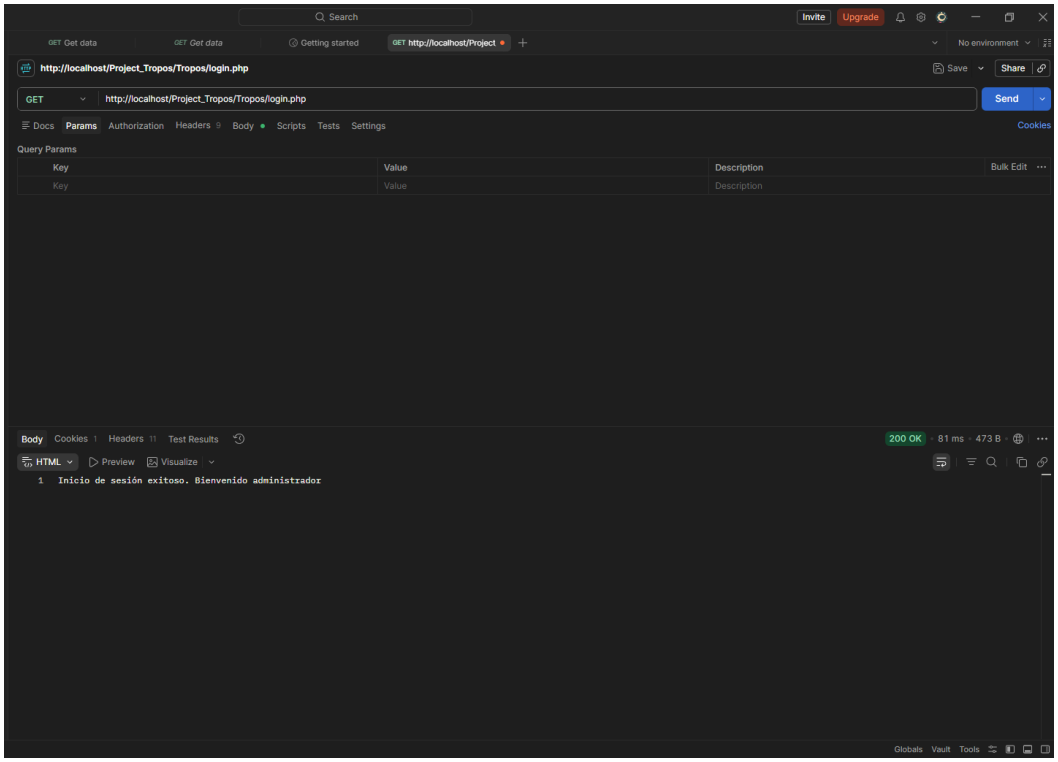
$sql = "SELECT id, name, password FROM usuarios WHERE email = ?";
$stmt = $conn->prepare($sql);

if (!$stmt) {
    die("Error crítico en la base de datos: " . $conn->error);
}

$stmt->bind_param('s', $email);
$stmt->execute();
$result = $stmt->get_result();

if ($user = $result->fetch_assoc()) {
```

```
    if (password_verify($password, $user['password'])) {
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['user_name'] = $user['name'];
        echo "Inicio de sesión exitoso. Bienvenido " . $user['name'];
    } else {
        echo "Contraseña incorrecta";
    }
} else {
    echo "Usuario no encontrado";
}
?>
```



Control de Errores y Depuración Técnica

Durante la fase activa de desarrollo, testing y aseguramiento de calidad del software para Tropos, se presentaron importantes retos técnicos asociados al despliegue local del entorno Apache y MySQL en XAMPP, el direccionamiento de puertos y la validación de sintaxis estricta, lo que requirió una depuración exhaustiva:

El primer error crítico detectado fue el fallo de ejecución en el script de registro inicial, representado por el error Fatal error: Uncaught Error: Call to a member function bind_param() on bool in crear_admin.php. El origen de esta falla se ubicó en un desajuste o inconsistencia nominal entre la consulta SQL del backend PHP (la cual realizaba una llamada sintáctica al campo nombre en español) y la columna definida físicamente dentro del esquema de la base de datos MySQL (la cual estaba definida bajo el identificador en inglés name). Al enviar una columna inexistente, el motor de la base de datos rechazaba la consulta provocando que el método nativo $conn->prepare() fallara de manera silenciosa devolviendo un valor booleano falso, sobre el cual no se puede ejecutar la función bind_param(). Al depurar la consulta y unificar el campo bajo la clave física name, se resolvió el conflicto de persistencia.

El segundo obstáculo se manifestó en la respuesta del endpoint de autenticación, donde se presentaban de manera recurrente las advertencias de tipo Warning: Undefined array key "email" y "password" en login.php. Esto ocurría porque al testear la petición mediante Postman, los datos estaban siendo direccionados mediante la pestaña de parámetros de URL (*Params*) o como encabezados (*Headers*), lo que generaba un objeto vacío dentro de la superglobal $_POST de PHP. Para resolver esto y dotar a la API de Tropos de robustez profesional, se codificó un bloque dinámico de detección híbrido que lee el flujo del sistema nativo php://input

para decodificar datos serializados en crudo en formato JSON, eliminando por completo cualquier advertencia de sintaxis en el servidor y permitiendo el procesamiento transparente de datos transaccionales de inicio de sesión.

Conclusiones

La estructuración de la capa lógica de Tropos mediante PHP 8 y MySQLi ha significado una evolución decisiva en el proyecto, superando de manera contundente las limitaciones arquitectónicas intrínsecas a las vistas iniciales maquetadas de forma estática en HTML5 y CSS3. La construcción de un canal modularizado a través del archivo db.php y el desarrollo de endpoints estandarizados de backend representan un hito técnico, permitiendo que la interacción del usuario final no se limite a la visualización pasiva de productos, sino que incluya el control dinámico de sesiones seguras y la persistencia consistente de sus datos dentro de un motor relacional local.

La aplicación de metodologías profesionales de debugging y la adición del procesamiento de flujos asíncronos y formato plano JSON demuestra que el backend de Tropos está técnicamente maduro y preparado para integrarse de forma fluida con interfaces SPA construidas sobre la biblioteca React. Al mitigar errores críticos de correspondencia de base de datos y advertencias de keys indefinidas, el código fuente adquiere una solidez y legibilidad óptimas, alineándose estrictamente con la guía de estilo de desarrollo limpio aplicada en el control de versiones en GitHub. Esta arquitectura transaccional no solo garantiza el cumplimiento de las exigencias del programa formativo ADSO del SENA, sino que cimienta las bases conceptuales para la incorporación futura de módulos transaccionales de comercio electrónico reales, tales como pasarelas de pago externas, control automatizado de inventarios y despliegues productivos escalables en la nube.

Bibliografía

- Airbnb. (2026). *Airbnb JavaScript Style Guide: Nomenclaturas y Estándares Limpios de Programación*. Recuperado de https://github.com/airbnb/javascript
- Mozilla Developer Network (MDN). (2026). *Manejo de Sesiones y Control de Estado en Aplicaciones del Lado del Servidor*. MDN Web Docs. Recuperado de https://developer.mozilla.org/es/docs/Web/HTTP/Session
- Postman Learning Center. (2026). *API Testing, Requests and Environments*. Documentación oficial de Postman para validación de servicios web. Recuperado de https://learning.postman.com/docs/sending-requests/requests/
- Servicio Nacional de Aprendizaje (SENA). (2026). *Material de formación: Diseño y desarrollo de servicios web; Herramientas de versionamiento y estándares de codificación*. Plataforma Virtual Territorium (Consulta asociada a las sesiones sincrónicas del tecnólogo).